School of Computer Science
FACULTY OF ENGINEERING AND PHYSICAL SCIENCES

UNIVERSITY OF LEEDS

Final Report

Creating an Immersive Basketball Experience in Virtual Reality

Pío Cañas Toimil

Submitted in accordance with the requirements for the degree of
Computer Science (High-Performance Graphics and Games Engineering)
MEng, BSc

2024-25

COMP3931 Individual Project

The candidate confirms that the following have been submitted*:*

Items	**Format**	**Recipient(s) and Date**
Final Report	*PDF file*	*Uploaded to Minerva (28/04/2025)*
Participant consent forms	*DOCX file*	*Uploaded to Minerva (28/04/2025)*
Link to online code repository	*URL*	*Sent to supervisor and assessor (28/04/2025)*

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

Summary

This project aimed to develop a realistic and immersive basketball shooting experience in Virtual Reality, targeting standalone hardware like the Meta Quest 2. The main objectives were to simulate accurate throwing mechanics, implement a responsive scoring and UI system, and create engaging game modes that reflect the feel of real-life basketball. Built in Unity using the XR Interaction Toolkit, the application features custom shot assist logic, modular game architecture, and performance-optimized design. User testing confirmed strong immersion, with high scores for realism and responsiveness. The project successfully demonstrates how VR can deliver a skill-based sports simulation, laying a foundation for future expansion into multiplayer, advanced mechanics like dribbling, and AI opponents.

Acknowledgements

I would like to thank my supervisor Dr Rafael Kuffner dos Anjos for his help and assistance throughout the project and to Professor Steven Noble for giving me advice on the report. And a special thanks to my housemate Mr. Finn “Cu” Bradley for lending me his Quest 2.

Table of Contents

1 Introduction and Background Research

1.1 Introduction

Over the past decade, virtual reality (VR) has moved from a futuristic concept to an accessible and widely used technology. Its ability to simulate real-world activities in a virtual environment provide a level of immersion that no other technology can, giving users the chance to interact with digital worlds in ways that feel natural and engaging.

This project explores the creation of a virtual basketball experience, where users can step into a digital basketball court and shoot the ball using a VR headset and controllers. The main goal is to evaluate how immersive, fun, and realistic the game experience is, and how well users engage with it in comparison to both real-world basketball and other basketball video games.

To effectively develop and test the experience, the game was built in Unity for VR, specifically for the Meta Quest 2 headset. Users can play the game and then provide feedback through ratings and evaluations.

1.2 Literature Review

1.2.1 VR in Sports and Basketball Applications

Virtual Reality (VR) has gained popularity lately within professional sports organizations and among everyday users at home for entertainment.

1.2.1.1. VR for Sports Training (Non-Gaming)

For athletes, it provides a safe, repeatable and customizable way to replicate in-game situations where the coach can run drills and a player is actually put inside the moment. This allows for a more precise measure of their decision-making since they can only see what they would see in real life and the technological aspects of the simulation can be leveraged to gather statistics about said drills or to customize them and easily transfer and repeat between players. It is also helpful for injured athletes going through rehabilitation because it “provides safe, controlled environments for injured athletes to maintain skills and conditioning while physical rehabilitation progresses” (1).

The technology also helps identifying movement patterns that might predispose athletes to certain injuries, like head concussions (2), and help prevent them. Athletes suffer from

cumulative load or wear and tear since the peak of athleticism is increasing every generation and “overload in individual athletes using pragmatic load management is crucial to protect athletes’ health” (3) a good tool to assist this is using VR for training tactical methods where a strain on the body isn’t essential.

Elite sports performance demands top tier cognitive reaction time and hand eye coordination skills. VR training can easily target these aspects of training which can be harder to replicate in real life training due to physical limitations of such. “The cognitive load during these drills can be quantified using a modified Hick’s Law equation”:

$$RT = a + b\log_2(n + c)$$

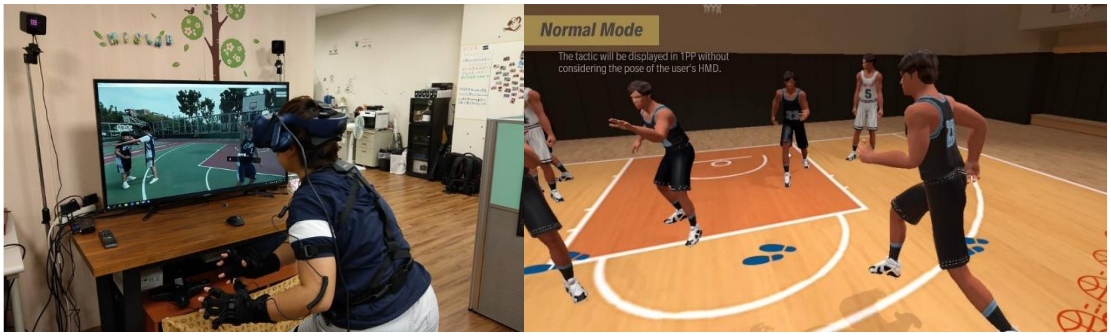
“Where *RT* is reaction time, *n* is the number of stimulus-response alternatives, and *a*, *b*, and *c* are constants that can be measured and improved through VR training” (1). Having this in a virtual environment can really quantify players progression. Alongside an immersive environment for repetition allows athletes to refine their skills with immediate feedback mechanisms telling them minor adjustments to their movements that can greatly improve their game.

Most studies found VR shooting training caused an increase in the shooting percentage or an improvement in the shooting mechanics of participants (4). A specific study’s results display that “the highest shooting percentage increase is about 14%, and the lowest is around 4%” (5). This highlights the purpose of the project and why it’s important that it really feels like real life basketball shooting.

An example that is used in the league (NBA) is VReps. It been tested on elite All-Star calibre players like Jalen Williams (Figure 1).

Figure 1: Jalen Williams, the 12th overall pick by the Oklahoma City Thunder in the 2022 NBA Draft, tests out VReps (7).

A big factor that distinguishes elite NBA-level players from standard professional basketball players is the mental understanding and the strategy behind it (6). Through repetition and set-up of these in game playbook situations the coach can really develop their players whilst not having to coordinate a whole other group of players to simulate said plays.



Figures 2 and 3: Displaying a VR training assistant for offense developed by students at NCKU (8)

Beyond training and rehabilitation, the realism and immersion offered by VR are essential for compelling and enjoyable virtual sports experiences. When designed to replicate the mechanics, strategy and competition of real-world play, VR systems not only work as training tools but also as sources of entertainment. The rewarding sense of progression that comes from improving a skill can be just as satisfying in a virtual game as it is on the court. This blend of fun and functionality works perfectly for VR basketball games, which aim to strike that same balance between real mechanics and engaging gameplay.

1.2.1.2 Existing VR Basketball Games

The Meta Quest Store hosts several VR basketball games that balance immersion and realism with fun and engaging mechanics. Allowing players to performs dunks which the average person can't do in real life or to play online matches with and against friends. Ranging from casual games focused on entertainment to competitive games with a real learning curve where you can progressively get better at.

There are three basketball games in the Oculus store which share the great majority of the market:

- **Gym Class VR.** A game that received investment from NBA franchise Golden State Warriors and NBA legends such as Kevin Durant and Andre Iguodala and even has official NBA licensing (9), is bound to provide one of the best VR basketball experiences out there. Developed by IRL Studios, Gym Class VR focuses on delivering a realistic basketball experience by emphasizing accurate physics and full-body tracking (10).

- **BIG BALLERS – VR SPORTS.** This game isn't just limited basketball, it also has baseball and American football modes. Its designed to be a multiplayer experience with lobbies of up to 10 players to have a full court 5v5 game. It leans more on the arcade aspect of the experience allowing for flashy skins and crazy dunks (11).
- **Blacktop Hoops.** It delivers a streetball-like experience with cartoon aesthetics that match and the ability to perform crossovers and fadeaways. It needs a good amount of space to be played to allow full immersion and to perform the tricks which make this game so exciting (12).

These VR basketball games demonstrate the potential of virtual reality to redefine how people experience sports from home. Beyond realism and immersion, they offer a sense of physical immersion that traditional sports games can't replicate. Unlike being limited to a monitor and a controller, players are fully involved in the actions of playing basketball, like shooting, jumping, dribbling all with real life movements. The social aspect of VR also adds a layer of and excitement, if you're playing with or against your friends, it feels a lot more tangible.

1.2.2 What Makes VR Feel Real?

The realism of a virtual reality experience comes from a combination of four crucial technologies (13):

- Visual, aural and haptic displays
- Graphics rendering system
- Body tracking
- Database construction and maintenance system

Knowing how and why they work is essential for creating an immersive application in VR. Immersion was described by *Palmer* as "the degree to which users of a virtual environment feel involved with, absorbed in, and engrossed by stimuli from the virtual environment" (14). The essence is about transporting the user into an experience where he feels he is an actual part of. This illusion of presence is made possible by tricking your brain to feel certain sensations that are caused in real life interactions. Whether it's how you collide with parts of the world and feel a response in the controllers or when your real life motions are instantly transported allowing you to interact with the space on much deeper level. Users can feel an effect on their balance and sometimes simulation sickness, which is one of the drawbacks of the technology. One of the ways VR tricks your brain is by displaying slightly different images for each eye and creating an artificial sense of depth by using stereopsis (15).

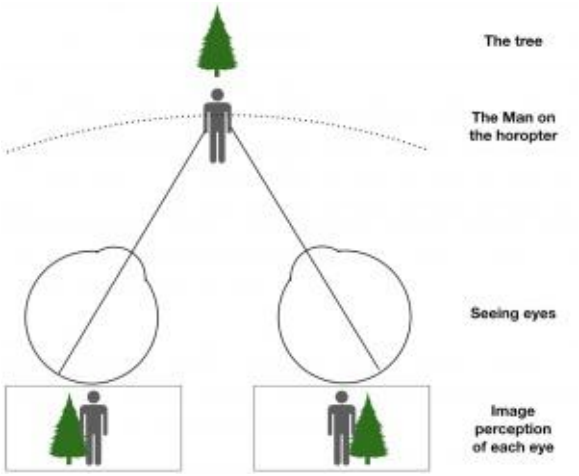


Figure 4: Diagram showing the difference in image perception of each eye before the brain process it to one singular image (15)

Stereopsis is defined as “the visual process of perceiving depth and three-dimensional space, achieved through binocular vision” (16). It’s essential to give the sense of depth on a ‘flat’ screen so that the players vision can easily adapt to the display view and act as a simulation of how they see in real life. Game engines such as Unity handle this by rendering the two offset camera views for each eye and rendering the scene from both perspectives simultaneously.

1.2.3 How VR Systems Capture and Translate Movement

A fundamental aspect of immersion in virtual reality is the accurate tracking and translation of user’s movement into the virtual world. Modern VR headsets use six degrees of freedom (6DoF) tracking. This allows for complete spatial interaction because it combines translational (x, y, z) axes and rotational (pitch, yaw, roll) axes that represent the ability to rotate on x, y and z respectively (17).

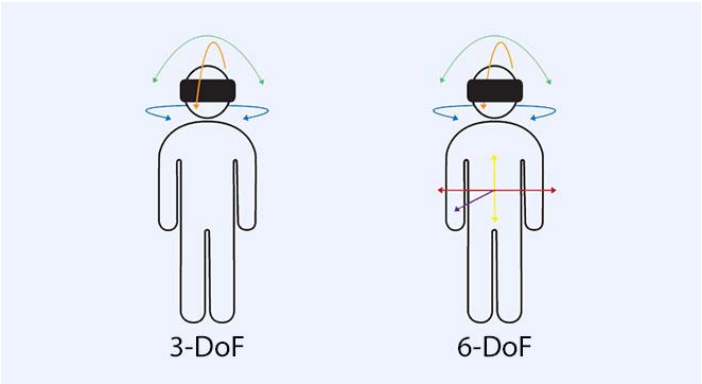


Figure 5: Diagram highlighting the difference between 3-DoF and 6-DoF (18)

Tracking and Data Collection

Most headsets use inside-out tracking (19), where built-in cameras track the users surroundings and environment and also track the position and rotation of the headset and controller without needing external sensors. Other data tracked includes controller input such as touch and pressing buttons and triggers and even the grip-intensity on some advanced controllers. Some headsets use the cameras to track hand movement and joints so that the user can interact without a controller and using only natural hand gestures.

Translating Data into Interaction

Specific data needs to be translated their appropriate method of interaction to allow for immersion. Game engines process this raw tracking data through middleware such as the XR Interaction Toolkit in the case of Unity. For example, turning your head will in turn rotate the in-game camera and moving the controller will translate the players hand to their respective coordinates. These interactions are made more realistic and believable by simulating real world physics so the players velocity and force will affect the games environment accordingly. If a player grabs an object and throws it at a slow speed, then that object won't fly as much as it would if the player throws it with a lot more force. This is calculated using the controllers velocity and it gives the player a real sense that their actions have realistic impact.

Understanding these techniques and how the VR framework works is essential for developing any kind of simulation in virtual reality.

1.2.4 Simulating Throwing in VR

An essential user interaction for the development of this project is throwing. It involves a lot of different data like the tracked data from the controller such as the rotation, velocity, position and point of release. It also involves the data on the game object being thrown including the position, velocity, rotation, mass, external forces (gravity, air resistance, friction) and its physics material which influences on the bounciness of the object. Translating this taking into consideration the player doesn't throw a physical object but just lets go of a trigger button means the challenge is in tricking the player that their actually holding and throwing something with real weight. Without the physical feedback of a ball leaving the hand, users rely fully on audiovisual cues to judge the quality of the throw.

However, current VR systems often struggle to replicate the realism and consistency of real-world throwing. A study by Zindulka (20) found that users were twice as accurate and 2–3 times more precise in real-world throwing tasks compared to VR. Participants specifically struggled with vertical and distance accuracy, largely due to difficulty timing the release using controller triggers rather than natural hand opening (21). This highlights the natural and expected limitations of the hardware, for example how the controller is always one specific shape and game objects aren't going to have the same grip or form to hold on to. The grabbing position the hand has on an object in real life affects the throwing mechanics so developers need to compensate for the lack of proprioceptive (the body's ability to sense its position and movement in space) feedback. This can be done by providing assist on the users throws or adjust their release velocity in real time.

The point of release (PoR) is a key variable that determines how natural and accurate a throw feels. Ghaemmaghami (21) investigated various PoR mechanics in VR, such as manual controller release, threshold-based automatic release, and on-body tracker methods, and found that while automatic systems produced the most accurate results, users preferred the manual release due to perceived control. This aligns with how users feels when playing non-VR video games where the control and game feel is more important to them than just being straight realistic. There needs to be a balance between what is pure realism and what makes a game enjoyable.

1.2.5 Summary of Key Design Considerations

The design of this project must focus on accurately simulating basketball shooting mechanics within the constraints of VR. Key elements include realistic ball trajectory, consistent throw detection, and immediate feedback to ensure user immersion and enjoyment. Literature shows that perceived realism in VR throwing improves with velocity-based simulations and applying assist to compensate for hardware limitations.

This review helped establish the core design requirements: intuitive interaction, realistic physics, and immediate feedback to the user. These give reason to the implementation decisions discussed in Chapter 2, supporting the overarching aim which is to deliver a responsive, immersive VR basketball experience that balances realism with fun. The project's deliverables include a fully playable VR prototype featuring multiple game modes, and user testing results.

2 Methodology

The purpose of this chapter is to outline the design, development, and planning methodology used in the making of the game. The focus is on how and why different technical tools, design decisions, and workflows were used to achieve the project's goal: creating an immersive and responsive basketball shooting experience in virtual reality.

A significant technical challenge addressed in this project was creating a realistic simulation of basketball throwing physics in a way that feels natural. So development was focused on immersion, realism, usability, and performance.

The sections below detail the specific tools and technologies used, the structure of the application, and the workflow followed.

2.1 Technology Stack

2.1.1 Unity VR

Unity was chosen as the development platform due to its native support for VR through the XR Interaction Toolkit, its physics engine (Nvidia PhysX), and ease of deployment to the Meta Quest 2. The free academic license and easy to use asset pipeline also made it ideal for prototyping and constant testing.

Unity's XR Interaction Toolkit (XRIT) abstracted low-level input handling, allowing intuitive player interactions such as grabbing, throwing, and menu selection without requiring custom controller bindings. These features enable iteration of gameplay mechanics like shooting and grabbing while maintaining performance with standalone VR headsets.

The built-in physics engine handled real-time collision and trajectory simulation, which was essential for realistic ball behaviour. By leveraging Unity's C# scripting and Rigidbody physics components, the project implemented personalised mechanics such as shot velocity, spin, and assist logic.

2.1.2 Meta Quest 2

The project was made as a standalone build targeting the Meta Quest 2, meaning that it runs locally on the headset and doesn't need to be connected to a PC.

Some of the technologies and API's required to make this work where:

- ***Android Build Support module in Unity*** that allows exporting projects to compatible APK files for the headsets OS.

- ***OpenXR Plugin*** which is a standard interface to access all types of VR headsets from different hardware, so not only the Quest 2.
- ***Android Debug Bridge (ADB)*** for installing the game's APK (its executable file) and access the device during debug.

While the Meta Quest 2 is widely known for its popularity and accessibility, those aspects are discussed in the literature review (Chapter 1). Here, it is important to note that its technical constraints heavily influenced the design decisions around physics complexity, and interaction responsiveness.

Version Control & Build Management

Git was used for version control. Unity's .meta files and scene assets were tracked using .gitignore best practices. A GitHub Kanban board was used to loosely track tasks and Unity Cloud Build was also used initially for version control

Other Tools

- **Blender**: For basic models like the rim and custom mesh colliders to detect 3 pointers
- **Unity Asset Store / Vecteezy**: Used to source environment models, textures, and props
- **Quest Developer Mode**: Used for real-time debugging and testing of the program

This stack provided the core infrastructure needed to iterate quickly, test frequently on actual hardware, and maintain the projects structure.

2.2 Game Design and Implementation

2.2.1 3D Assets and Environment

The virtual court design:

It was made following a YouTube tutorial (22) by a Blender content creator called Magevo Studio. In the video he goes step by step on creating a 3D model of the basketball court including lights, hoop and net, backboard and also the walls around the court. This was specially convenient due to the model being open license and also this allowed for easy manipulation of all the objects in the model and their material to be customised.

Different parts of the court interact with the ball in different ways, for example when a ball bounces on the ground it is expected to bounce almost to the height it was dropped from but if it hits the backboard, it is expected to absorb most of the impact. Having different objects also allows for custom textures that would fit the project accordingly. These choices aid the

goal of achieving realism and immersion because it allows for easy physics interactions of the ball with its environment whilst being able to have aesthetics that match the feel intended for the experience.

The outside environment of the court including the sun, which provides lighting to the court, and the buildings on the horizon and on the proximity of the court where all from Unity's Asset Store. These free to use models where important to provide the user a sense presence in the experience and not as if they were thrown into an empty and uncanny world with nothing but a basketball court.

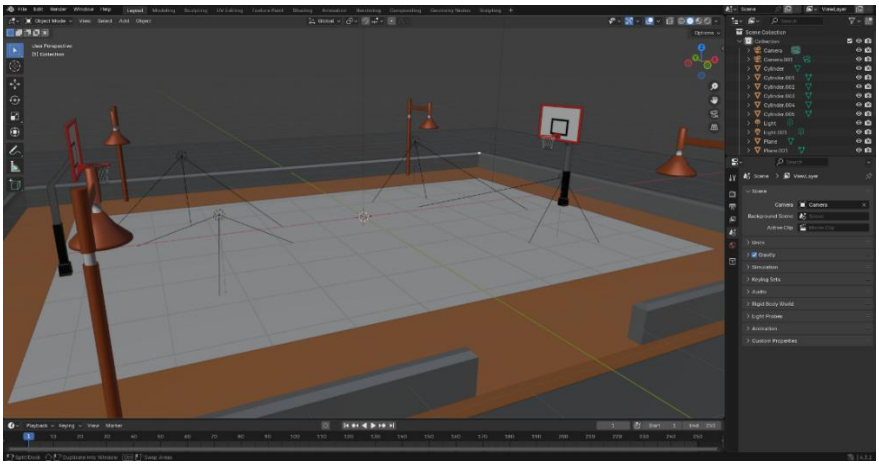


Figure 6: Screenshot of the basketball court model in Blender (adapted from Magevo Studio tutorial).

Basketball and virtual hand:

The models for the basketball and the virtual hand are from the Unity Asset Store that come readily available with Unity's VR tutorial scene. That is also including their textures and their colliders.

A virtual hand is necessary so that the player can make that connection from their movement to what they can see, allowing them to feel like they are there and that they can interact with the environment using said hands.

Textures where picked off of stock image libraries that provide license-free vectors suitable for personal or academic projects. They were select taking into consideration their interaction with the lighting of the scene

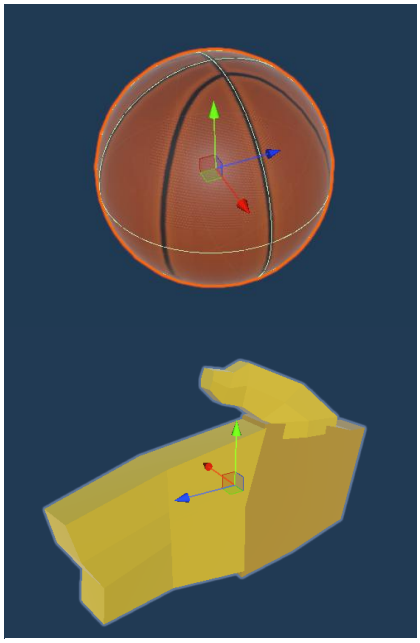


Figure 7: Models of the hand and ball

and how well they fit that environment.

All of these assets are suitable because they are readily available and optimized to run smoothly without an unnecessary amount of polygons that would greatly diminish the performance of the application when the VR is trying to render it.

2.2.2 Gameplay Mechanics

2.2.2.1 Shooting

The core mechanic of the experience is shooting, which needed to feel intuitive, realistic, and responsive. The system was designed to reward textbook shooting form and penalise erratic or overly casual throws, replicating the skill ceiling of real-life basketball.

The main challenge is translating that real life shooting motion into VR taking into consideration all the limitations that the medium brings such as the controller obviously not being shaped like a ball and the fact that when a player does the motion they don't let go of the controller like they would with a basketball. The engine simulates the release by capturing the velocity, direction and angle of the controller and the exact point where the player releases the grab button.

In this design XRIT is used to detect all these values at point of release and apply them to the basketball's Rigidbody component. In Unity, a Rigidbody enables a GameObject to behave according to real-world physics — it allows objects to respond to forces, gravity, and collisions through the NVIDIA PhysX engine (24). This makes it essential for simulating realistic motion and interaction within the virtual environment. To maintain realism these forces are adjusted accordingly to simulate factors such as:

- **Velocity:** The ball takes the controller's linear velocity to simulate throw strength.
- **Arc and Spin:** Angular velocity affects trajectory, mimicking wrist flicks.
- **Assist Logic:** A custom assist system subtly adjust the shot's direction based on distance. This uses a correction factor to smooth out small inaccuracies when throwing with the Quest 2's controller.

The ball's Rigidbody uses tuned drag, gravity scale, and mass values to replicate the real-world feel of a 600g basketball (vs the ~150g controller). Shooting from greater distances requires the player to generate more force, mirroring the effort needed in real-life basketball. Players use their entire body to produce the power needed for a high-arching shot with a regulation basketball so that same effort needs to be translated into the virtual world.

Rather than aiming for a perfect one-to-one throw replication, the system is designed to reward realistic shooting techniques and offer players a clear path to skill progression. It requires practice and precision, echoing the learning curve of real-world shooting.

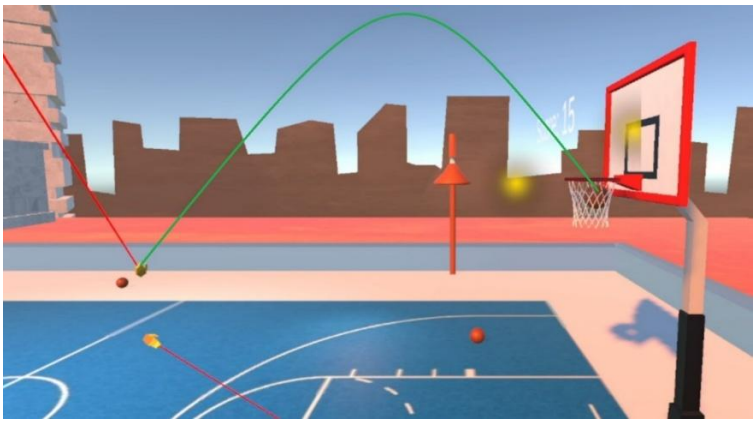


Figure 8: Drawn on Gizmo showing the trajectory of a made three pointer

2.2.2.2 Dribbling

The goal was to allow players to replicate basic dribbling motions using only natural hand movement with the controller, without relying on button presses.

Early design focused on using physical hand gestures to trigger a bounce, aiming to create a fluid, responsive feel. A key requirement was to keep the interaction grounded in realism while remaining simple enough to integrate with the existing physics-based systems.

However, due to the technical complexity of replicating realistic dribbling behaviour, it was flagged as a lower-priority feature. While early design considerations were made, full implementation was deferred, and the focus was shifted to perfecting shooting.

2.2.3 UI and Player Experience

The **User Interface (UI)** in the VR basketball experience was designed to be simple and unobtrusive, focusing on conveying essential gameplay information without breaking immersion.

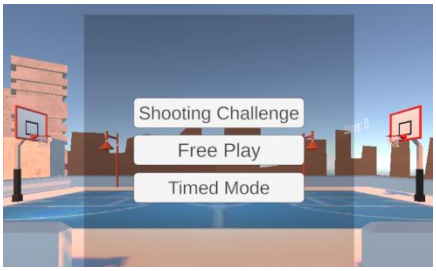


Figure 9: Main Menu with Game

- **Menu:** It appears at the start of the game, allowing the user to select game modes.
- **Score Display:** A live scoreboard updates dynamically as the player makes or misses shots. It shows the total score and is always visible within the court space.

Figure 10: Score and timer UI

- **Timer and Field Goal Efficiency (FG%) Display:** During timed game modes, a countdown timer is displayed. The FG% is also shown, representing how many successful shots the player has made versus attempted.
- **Sound Design:** Audio plays a key role in realism. Different collision sounds were used depending on where the ball hit:
    - **Rim:** Metallic and springy
    - **Backboard:** Dull thud to simulate wood
    - **Net (Swish):** Only the sound of net movement, no rim contact
    - **Bank shot:** Combo of backboard impact + net
    - **Made Shot:** Game-like sound effect to signalize scoring

All sounds were sourced from royalty-free sound libraries and synced with the physics collisions in Unity using AudioSource triggers.

2.2.4 Game Design

The game includes three main modes: **Shooting Challenge**, **Free Play**, and **Timed Mode**. Shooting Challenge guides players through a sequence of predetermined shots from various positions tracking accuracy (Figure 11). Free Play allows for practice and experimentation with no time constraints. In Timed Mode, players have to score as many points within 30 seconds.

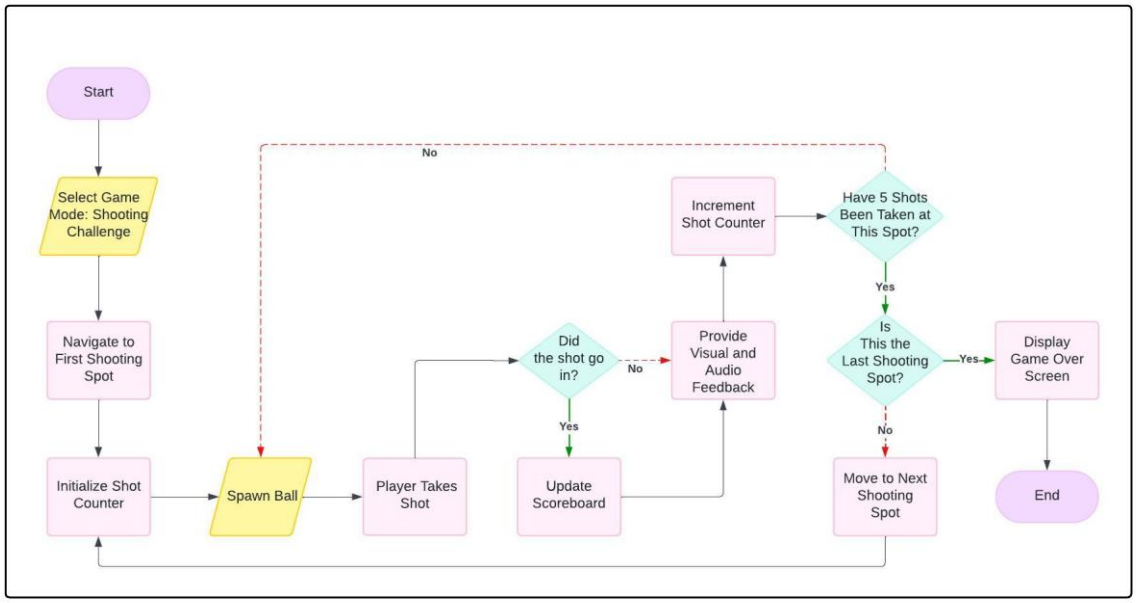


Figure 11: Shooting Challenge Flow – This diagram outlines the user's journey from menu navigation through the selected game mode, highlighting the different loops and conditions for completion or return to the menu

2.3 Project Management and Development Workflow

A hybrid development approach was used, combining structured planning from the Waterfall method with iterative improvements inspired by Agile practices. Early stages were mapped out in a high-level roadmap focusing on key milestones: shooting mechanics, scoring, game modes, and user interface.

Agile practices were followed, especially once the foundation was in place. After early prototypes of shooting and ball physics were created, they were tested repeatedly using the headset. Feedback from these tests helped recognise needed improvements, such as adjusting force calculations, collision detection, and hand tracking responsiveness.

This iterative cycle also extended to systems like UI design, score tracking, and game logic. For example, the scoring logic was initially designed as a simple point counter but was expanded to include field goal efficiency and shot types after testing showed that players wanted more feedback. These changes were not part of the initial plan but were incorporated mid-development thanks to the flexibility of Agile iteration.

Tools like GitHub and Unity Cloud Build were used throughout development for version control and collaboration. Regular APK builds were pushed and installed on the Quest 2 via ADB, allowing for fast deployment and testing. These continuous were essential in improving the mechanics and ensuring optimal performance on the VR's hardware.

2.4 Software Architecture and Data Handling

The architecture is modular and performance-focused to perform on efficiently on VR.

- **Object oriented design** was used to compartmentalize and allow scalability for different components. This means that in the future multiplayer could be implemented with several balls at the same time. Every GameObject has their own scripts that allow the gameplay system to interact with each other.
- **Event driven interactions** were essential for the triggering of different scripts such as updating when a player was inside the three point line or when a ball went through the hoop all using Unity's collision detection. Such events will then trigger a script that, for example, increments the score and displays it in the UI whenever a shot goes in.
- **Physics based data collection** includes converting player input into motion data (velocity, angle, spin) and pass it to the ball's Rigidbody component.
- **Score and statistics tracking,** as mentioned above they are triggered by events and other scripts to determine what type of shot the player took and how much its worth and then calculating the players field goal efficiency. The way the data

handling works was made in a way in which it could potentially be logged in a system with high scores for different users in the future and use that data to tailor the game to each users skill level.

The relationships between key components and scripts are illustrated in the UML Class Diagram (Figure 12), which highlights how modular elements like the scoring system, game modes, and XR rig interact with each other.

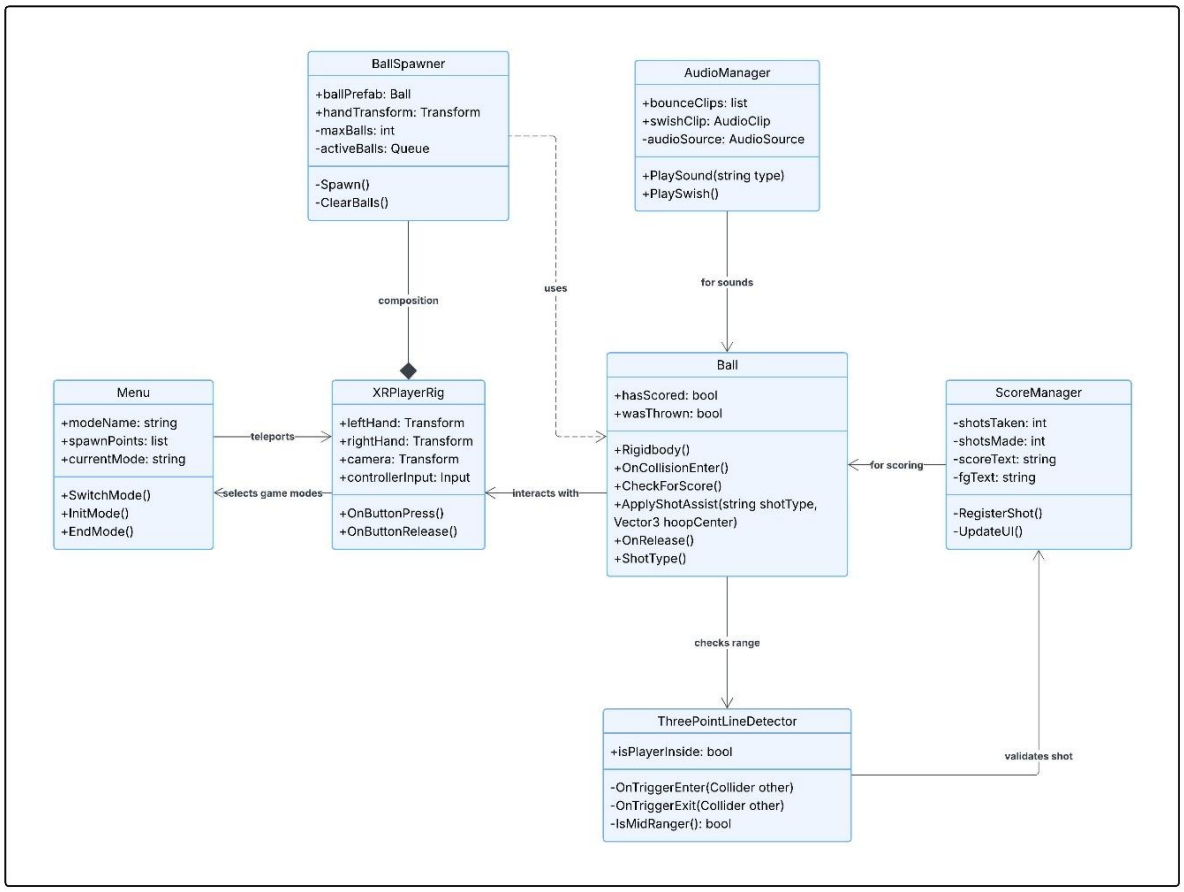


Figure 12: UML Class Diagram of Game Architecture

Each component (e.g., ScoreManager, Ball, XRRig) operates independently but communicates through Unity events or direct script references, enabling isolated debugging, easier scaling, and clear code separation.

This modular and event-driven architecture provided the technical foundation for implementing the core features described in the next chapter. Chapter 3 presents a detailed breakdown of how each component from throwing physics to game modes were implemented, tested, and iteratively improved based on feedback and performance goals.

3 Implementation and Validation

3.1 Project Setup and Development Environment

Unity Project Configuration

- **Unity XR Plugin Step.** OpenXR was selected and specific support for the Quest 2 was enabled by installing the Meta Quest Feature Group.
- **Build Configuration.** Texture compression was set to ASTC and compression method to LZ4 which was recommended by the plugin tool Meta XR Tools. Debug builds were created with logging enabled for profiling on-device.

3.2 Implementation of Core Features

3.2.1 Environment and Assets

The scene was constructed in Unity using a combination of Asset Store props, custom Blender models (court, rim, net), and license-free textures from libraries like Vecteezy and Pixabay. The 3-point line was implemented as a separate mesh with collision detection to identify shots taken beyond the arc. Environmental props such as background buildings and lighting assets were imported to improve immersion without impacting performance.

To optimise for standalone VR, the scene used baked lighting and static batching. They are techniques that reduce GPU load by pre-calculating illumination (lightmaps) and merging static meshes to lower draw calls (25). All elements were placed in world space to align with the XR rig and ensure proper physics interaction**.**

Figure 13: Final view of scene in Unity showing all 3D models and assets

3.2.2 Game Modes

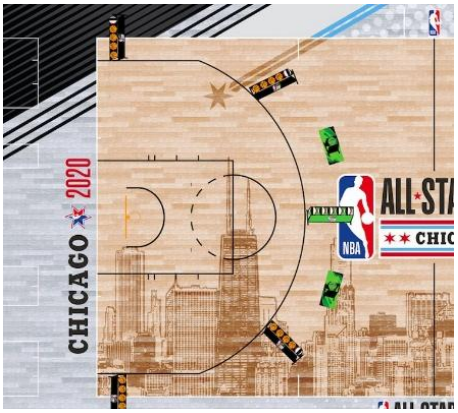


Figure 14: NBA three point contest shooting locations (26)

There are three game modes that the player can select from the main menu as soon as they load into the game. This was implemented by using a Game Manager object with a script that took in all the spawns for the games. For example, in Timed Mode and Free Play, the player gets spawned at half court and is allowed free range to move. On the other hand, when the player play the Shooting Challenge, their movement is disabled so that they can't move around and have to shoot at their designated spots. The way the different spots were implemented was using the following logic

```
public Transform[ ] challengeSpawns;
```

It is used to store the different shooting spots which were set on the physical court through scene view. Then the players XR Rig's locomotion component would be disabled to stop them from moving. After 5 shots at one spots they're teleported to the next one.

To select a game mode an On Click routine is activated whenever a player uses the controller to select a mode. Once its clicked, the menu disappears and a string containing the game mode is passed to the Game Manager script where it then initiates said mode.

3.2.3 Ball Mechanics

Grabbing:

It was implemented using the XR Interaction Toolkit. The ball has a XRGrabInteractable component that tracks when it is picked up or released. One major problem encountered whilst implementing the grabbing mechanic is how by default the point of contact where an object is grabbed is always the center. This is a problem with such a big object like a basketball since its larger than a hand and a person can't grab it that way. So to hold a basketball the hand is around the surface of the ball allowing the ball to rest on someone's palms and fingers. To work around this an empty object was created inside the basketball with the coordinates just on the surface of the ball making it the grab offset. This is set up in the XRGrabInteractable script of the ball that has the option to set an attach transform. Now the point that connects with the controller is on the outside of the ball and feels a lot more

realistic, this in turn affects the physics of throwing the ball since the force and spin changes once they are not being applied on the center point of the object. This was crucial for game feel and immersion and shots now can mimic real life with a lot more fidelity.

Figure 16: Left shows where the ball was grabbed on originally, right shows the ball being grabbed after adding an attach transform

Throwing/Shooting:

Initially, shooting the ball didn't work as it should. The lightest flick would launch the ball with a unrealistic amount of force and hitting any type shot felt way too difficult, the ball would always bounce out and getting a perfectly straight shot was hard.

To address this the mass of the ball was adjusted to match its real world inertia. Also a throw assist system was implemented. This involved making an assist radius for each type of shot, which meant that an assist for the shot was activated depending on how far away the ball is from the center of the rim. So for example a three point shot would have a higher assist radius since it's a more difficult shot in nature than a layup.

Each assist strength value was chosen through iterative in-game testing. During development, the assist strength and assist radius parameters were manually adjusted using Unity's inspector while observing shot outcomes in real-time. This trial-and-error approach helped find balance between realism and forgiving minor inaccuracies caused by the limitations of hand tracking or awkward VR throws. This improvement directly reflected how the assist system made the game more accessible and enjoyable without sacrificing realism as evidenced by field goal percentage taken from empirical testing. The FG% went from around 22% to more than 50% after assist was applied. These numbers were gathered from playing in the Shooting Challenge game mode for 5 times of 30 shots each.

Example: Throw Assist Code

```
        private void ApplyShotAssist(string shotType, Vector3 hoopCenter)
        {
            float assistStrength = 0f;

            //Calculate normalized direction from ball to hoop
            Vector3 directionToHoop = (hoopCenter - transform.position).normalized;
            //How far the ball has travelled since release
            float ballTravelDistance = Vector3.Distance(startingPosition,
transform.position);
            //Total distance from release point to hoop
            float totalDistance = Vector3.Distance(startingPosition, hoopCenter);

            //Assist factor grows with travel distance (prevents immediate correction at
release)
            float assistFactor = Mathf.Clamp01(ballTravelDistance / (totalDistance *
assistStartFactor));
            float scaledAssistStrength = assistStrength * assistFactor;

            //Apply assist to horizontal axes
            Vector3 newVelocity = rb.velocity;
            newVelocity.x = Mathf.Lerp(newVelocity.x, directionToHoop.x *
newVelocity.magnitude, scaledAssistStrength * Time.fixedDeltaTime);
            newVelocity.z = Mathf.Lerp(newVelocity.z, directionToHoop.z *
newVelocity.magnitude, scaledAssistStrength * Time.fixedDeltaTime);

            rb.velocity = newVelocity;
        }
```

Code snippet 1: Shot assist logic

- Dynamically chooses how much to help the shot based on distance and shot type.
- Only x and z (left/right, forward/back) are assisted — y (vertical arc) is untouched.
- Assistance increases the farther the ball travels, so it doesn't kick in too early.
- Helps preserve user intent while fixing small directional errors from VR tracking.

How Matf.Lerp(a, b, t) works:

It is a function with linearly interpolates between values a and b using a third value t, which ranges from 0 to 1. It is equivalent to:

$$(1 - t) * a + t * b$$

So in the context of this system it essentially changes the ball's velocity on the x and z axes from its current trajectory (newVelocity.x) to a velocity which will move the ball to the desired trajectory (directionToHoop.x * newVelocity.magnitude). The (scaledAssistStrength * Time.fixedDeltaTime) works a smoothing factor, so what percentage of assist is applied. Doing it like this ensures the ball's velocity gradually curves toward the hoop over time so there is no unnatural change of the path of the ball.

Bouncing:

Custom Physics Materials were applied to objects to simulate realistic bounce behaviour:

- Ball: High bounciness for natural dribbling and rim interaction
- Rim: Moderate bounciness to mimic springy, forgiving real-life rims
- Backboard: Low bounce for dull impact
- Floor: Tuned for realistic concrete behaviour

To replicate advanced shot types like the *"English" finish*—where opposite spin helps redirect a layup—additional angular velocity was applied to simulate the spin seen in high-skill basketball plays.

Spawning balls:

A circular queue system was implemented to handle ball spawning efficiently. Instead of endlessly instantiating new ball objects, which could negatively impact, a fixed pool of balls is reused. A maximum of five balls exist at any one time. When a new ball is spawned, the first ball in the queue, the oldest, is despawned and its space in the queue is reused. The spawn position is relevant to the players dominant hand and changes depending on what handedness they've selected in the settings. This method maintains performance and avoids clutter.

Dribbling (Prototype):

Dribbling was implemented as a prototype using downward controller velocity and hand-to-ball proximity to trigger bounce forces, with a cooldown to prevent multiple detections per bounce. However, several limitations arose:

- Difficulty syncing bounce timing with real motion
- No tactile feedback made contact feel laggy
- Clipping and lack of spin control made palm interaction unrealistic

- Sideways bounce control (for crossovers/in-and-outs) proved too complex

Due to these constraints, the system remained a prototype. A full implementation would likely require advanced gesture detection or machine learning to replicate real-world fluidity.

3.2.4 Scoring System

Figure 17: Three point line collision detector being displayed

The scoring system uses a trigger zone inside the hoop to detect made shots, registering only when the ball moves downward through the collider to simulate realistic scoring.

Three-point shots are identified using a hidden collider along the arc, created in Blender. If the player is beyond the arc when scoring, 3 points are awarded; otherwise, 2. When a shot is made, the score and FG% (Field Goal Percentage) are updated.

To enhance feedback, contextual sound cues are played based on hit type — including rim (metallic), swish (net only), backboard (wooden thud), and bank shots (combined audio).

3.2.5 User Interface and Game Mode Manager

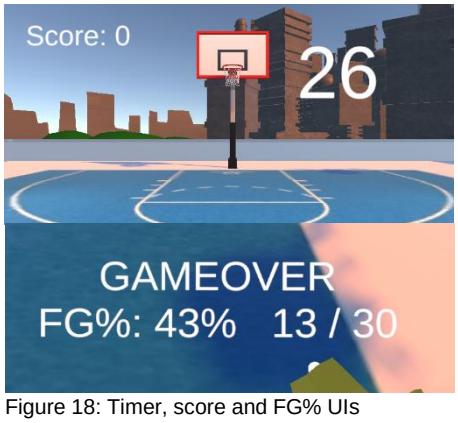


Figure 18: Timer, score and FG% UIs

The GameModeManager controls transitions between game states (Free Play, Timed, Shooting Challenge), updating score logic, timers, and reset conditions accordingly. To maintain modularity, Unity's EventSystem is used to broadcast gameplay events like successful shots, triggering independent systems for score updates, sound effects, and particles.

The UI employs world-space canvases to maintain immersion. The score and timer are fixed near the backboard for visibility without obstructing gameplay. The FG% display appears briefly at the end of timed challenges and is screen-locked to ensure players see their performance result. This layout avoids motion sickness associated with fixed overlays and supports intuitive player feedback in VR.

3.3 Validation of Implementation

Functional Testing

Feedback from early testing confirmed that improvements like throw assist, rim physics tuning, and adjusted grab points made gameplay feel more natural. The visual behaviour of the ball during shots and rebounds was compared to real-life expectations (e.g., arc height, rim bounces). This review process helped guide practical refinements for the gameplay.

Feature	Test Case	Result
Menu functionality	Select game modes and go back to menu	Pass
UI updates	Score + FG% after shot	Pass
Ball Spawn System	Spawns 6th ball (oldest removed)	Pass
Ball Spawn System	Balls can't be spawned outside of a game mode	Fail
Shot detection	Shoot from mid and from three point range and check scoreboard	Pass

Table 1: Part of feature and test case table showing failed and passed test

A few non-critical test cases didn't pass but, for this project, these were deemed acceptable and left for polishing in the future. The core features (shooting, UI, scoring) all performed reliably during testing. A detailed breakdown of testing can be found in the appendix.

Unit testing

To validate certain game play systems unit testing was performed through Unity's PlayMode Test Runner which runs all the scripts containing said tests and then outputs the results.

```
[Test]
public void SpawnBall_LimitsActiveBalls()
{
    // Spawn 7 balls – only 5 should remain
    for (int i = 0; i < 7; i++)
        spawner.Spawn();
    Assert.AreEqual(5, spawner.activeBalls.Count);
}
```

Code snippet 2: Test case for ball queue system

That is one example of a test case, ball spawn queue system, which passed meaning everything was working as expected.

Tests were also made for the Score Manager system to see if the score would increment correctly given different types of shots and the Shot Assist system to check that the assist was in fact improving the direction of the shot.

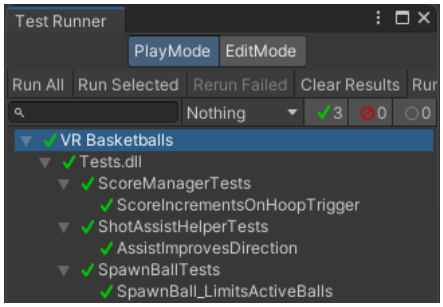


Figure 19: Unity Test Runner showing tests

Testing Performance

Performance was tested both in-editor (Unity Profiler) and on-device (Oculus Developer Hub). Three scenarios were evaluated: Free Play, Shooting Challenge, and menu transitions. Frame time consistently stayed under 16ms during normal gameplay (60 FPS target) and peaked at 33ms during stress tests with multiple balls and rapid score updates.

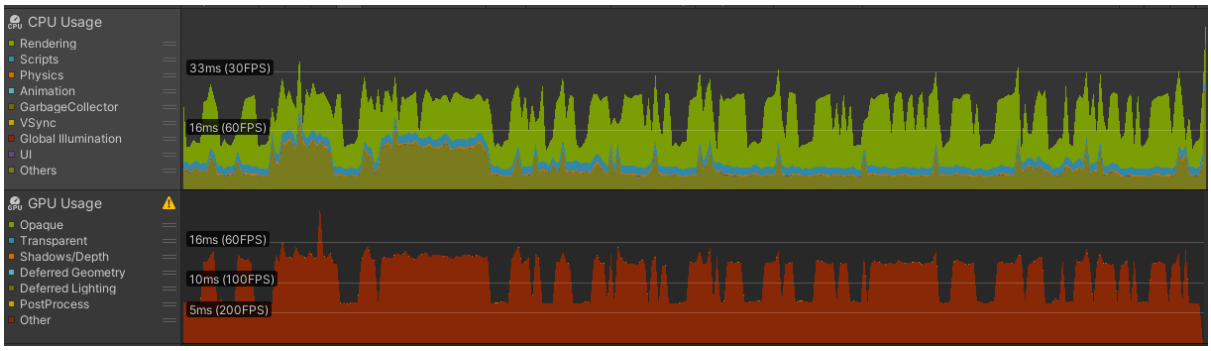


Figure 20: Performance testing results from Unity's profiler showing a few lag spikes

Figure X shows minor spikes observed during cable-linked testing, which disappeared in the standalone Quest 2 build. On-device performance exceeded 70 FPS in all modes with no lag during ball spawns or UI transitions, confirming that the physics and rendering pipeline remained optimised throughout.

User Feedback Process

To gather usability insights, a small group of early testers (3–5) were invited to use the prototype. Each tester was given a short set of instructions, followed by a brief post-session survey. The survey included both quantitative and qualitative questions such as:

- “Rate the realism of the shooting from 1–5”
- “How enjoyable was the experience overall from 1-5”
- “Was any part of the experience confusing or frustrating?”

More testers used the final prototype in a controlled setting, and their interactions were observed for signs of difficulty, hesitation, or confusion. Their written and verbal responses were logged for analysis in Chapter 4.

4 Results, Evaluation and Discussion

4.1 User Testing Setup

To validate the usability, immersion and overall enjoyability of the experience a user testing session was conducted. The primary objective of this testing was to assess how the core experience matched all its objectives and its aim of simulating an engaging and immersive basketball simulation in VR.

4.1.1 Objectives of the Testing

It aimed to evaluate:

- Immersion and feeling of presence
- How intuitive and easy to interact the experience is
- How responsive is the UI and audiovisual feedback
- How enjoyable the overall experience is

These metrics were chosen after being deemed essential during literature review and through the design and development stages.

4.1.2 Testing Environment

All testing was conducted using the prototype build deployed onto the Meta Quest 2 where the players were in a spacious empty room to allow for freedom of movement. A room with no obstacles is essential for the user to be able to interact with different levels of force and not hurt themselves or the equipment.

They were instructed to mimic their real-life basketball shooting mechanics as close as possible because earlier tests proved that to be most efficient and it is essential that that proves to be the case for all users since its attempting to be a realistic basketball simulator. Users were briefed before only on which sequence of actions they should take but not how until they would ask how to perform a specific action if they felt stuck.

Each user was told to get accustomed to the base mechanics on Free Play mode and once they’ve felt comfortable they could advance to play the Shooting Challenge mode where their shooting efficiency was recorded. Testers weren’t told how many times they had to do the Shooting Challenge as to have them attempt as many times as they wanted to beat their

own score would be a very good subconscious reflection of how engaged and how much fun they are having with the experience.

After they've attempted the Shooting Challenge, the shooting assist is removed and they are introduced to the program again to shoot around and assess the difference in feel.

All of this was being streamed onto a TV on the same room so that their vision and gameplay could be observed in real time for note taking or for assistant if the user needed any.

4.1.3 Participant Demographics

A total of 5 participants were chosen with varying degrees of experience regarding the use of VR and their basketball skills. This diversity allows to understand how easy it is for a new VR user to pick up the game and enjoy it and how engaging it is for a non-basketball player to interact with the experience.

ID	VR Experience	Basketball Experience
1	None	High
2	Some	High
3	Some	High
4	High	None
5	Some	Some

Table 2: Relation between participants and their experience in basketball and in VR

Participants where chose informally but with intention to cover the most different intersections of potential users interested in the experience.

4.1.4 Survey Design and Observational Method

A. Observation

During each testing session users were observed for:

- Any body language or vocal feedback that indicates confusion or frustration
- How quickly they adapted to the game controls and how much better they were getting at scoring the longer they played
- Whether they understood and how they responded to in-game feedback (game over screens, timers, sound and visual cues)

- If they'd feel frustrated after missing a shot and if they felt rewarded or enthusiastic after making on like how they would react to getting a better score.

B. Survey Feedback

After testing each participant completed a short survey including a quantitative and a qualitative aspect.

Quantitative Questions (1-5 scale):

- How realistic did the throwing mechanics feel?
- How immersive was the experience overall?
- How intuitive were the controls?
- Did the UI respond correctly and was it easy to understand?
- How enjoyable was the experience overall?

Qualitative Questions:

- Was any part of the experience confusing or frustrating?
- What would you improve or add?
- Did it feel like real basketball? Why?

All user responses are anonymous and their ethical approval and consent were obtained in accordance to the University of Leeds' participant testing policies. Both a Participant Information Sheet and Consent Form were provided and signed prior to participation.

4.2 Results from Testing

The findings were divided into measurable metrics, user opinions and observations. Results were compared against the project's goals: realism, intuitive interactions, immersion with good and responsive UI and feedback and overall enjoyment.

4.2.1 Quantitative Survey Feedback

Question	Average Score (1-5)
How realistic did the throwing mechanics feel	4.2
How immersive was the experience overall	4.8
How intuitive were the controls	4.0
Did the UI respond correctly and was it easy to understand	5.0
How enjoyable was the experience overall	4.8

Table 3: Quantitative survey and the average scores

Observations:

Realism and immersion scored excellently high scores meaning those goals were met successfully. Controls received the lowest score which might be due to the fact that there is a substantial learning curve, just like the real life sport would. So initially a few players felt frustrated until they got the grasp of the shooting mechanics. UI responsiveness received a perfect score completely meeting the aim set beforehand. And lastly, overall enjoyment received a 4.8 reinforcing the project's purpose of delivering a rewarding skill-based experience.

A few responses by users included:

"Scoring felt satisfying, specifically when hitting a swish" – when talking about what they enjoyed.

"Figuring out how much force was needed when shooting from deep was a bit hard" – when talking about what was confusing or frustrating.

"Allow to play against other people or bots" – when suggesting improvements.

Overall the project succeeded in delivering an immersive and fun basketball experience where users are challenged and can still enjoy a smooth gameplay.

4.3 Discussion

4.3.1 Strengths of the Implementation

The biggest strength of the project is the successful delivery of a highly immersive and realistic shooting experience. This is backed by both the qualitative and quantitative user feedback. Scoring highly across all 5 categories where users where questioned on, confirms that the goal of translating the real life shooting motions from basketball into virtual reality was well executed. This was achieved by integrating Unity's physics engine, adjusting the balls Rigidbody and its physics material, and developing a fine-tuned shot assist mechanic that provided that layer of help that the players needs due to the limitations of the hardware. These combined to produce realistic trajectories that followed the user's intentions whilst leveraging the highest capabilities that virtual reality headsets and controllers can provide.

Another notable strength, which received a perfect score in the user study, was the functional and intuitive UI. The layout and world-position allowed the user to be engaged in the game given important insights whilst not being obstructive and supporting the gameplay providing instructions and feedback without distracting the player. This also in turn favoured the aesthetics which matched the game feel and the game modes and flow were intuitive and appealing.

It is noted that game performance on the headset is crucial for immersion. This was noticed during prototype testing, when the Unity play mode would naturally lag, any user would automatically be sucked away from feeling present in the experience. This happens because in real life people don't experience such problems as lag or the program slowing down. That proves that polishing the game and ensuring it runs as smoothly as possible is a non-negotiable when it comes to transmitting the feeling of immersion. To make sure this happens, memory management, graphics rendering, game physics and other components of the project were designed and implemented with performance on the standalone VR headset in mind. This was ultimately a major strength of the project where no user experienced any technical issues which would've immediately ruined the feeling of presence.

Lastly, the project's architectural modularity, with isolated scripts for gameplay components (e.g., ShotMaker, ScoreManager, UIHandler), enabled maintainability, testing and scaling. Systems like the audio and particle feedback were decoupled from the main logic, using Unity's event system to streamline cross-component communication and simplify future updates or debugging. This also helped when performing unit tests to ensure code quality all around.

4.3.2 Limitations and Challenges

While the core shooting experience was well received, several limitations were to be expected from developing in virtual reality, particularly on standalone hardware like the Meta Quest 2, introduced challenges that impacted both design scope and realism.

Controller limitations

Although the controller and its tracking provides an incredible experience in VR it obviously comes with its limitations. A big one is the fact that a controller is always going to be in the shape that it is. That means there is really only one way to hold it and in a world where you can pick up all kinds of objects with different sizes and shapes almost always the real world hand grip won't match the expected one for such virtual object. This is specially apparent with a basketball, where its 6 times heavier than the controller and where the average person can't realistically palm a ball so they need both hand to hold it. It is really hard to replicate the hand grip because in basketball you're not really grabbing the ball and this in fact affects the way users throw the ball massively because it is not possible to add the iconic backspin with the follow-through from a shot. Other certain fine motor cues like wrist flicks or fingertip roll-offs can't physically be captured limiting the amount of moves available.

Dribbling complexity

The initial idea was to simulate dribbling using only natural motion and no button presses, relying only on downward controller velocity and the proximity to the ball. However the lack of precise hand tracking and the delay it naturally takes to update the physics loop makes it very difficult to do hesitation or to pick up the ball from a dribble. A lot of modern games with functional dribbling systems use AI models to predict certain moves given the hand motion and not a coded standard collision interaction. Adding cooldown to the dribble helped with creating a sense of rhythm to the dribble but there was no real way to communicate tactile impact which is essential for a realistic simulation for basketball dribbling. This broke the immersion and made it feel disconnected since it didn't bounce as expected or clipped through the hand entirely. As a result the mechanic was ultimately side-lined in favour of improving the rest of the application.

Limited feedback and realism constraints

In a real-world setting, physical feedback from the ball, court, and backboard informs how a player adjusts their shot or movement. In VR, the absence of haptic resistance or physical touch when touching a ball in movement introduces a disconnect from the immersion. While audio and visual feedback emulate the dribbling and bouncing on different materials, these cues can't fully replicate the sensation of having a ball with real weight leave your hand and feeling the resistance and grip of that basketball. This is particularly evident when the way a player brings up the ball for a layup fully affects how the shot comes off their hand and these subtle contact cues are not easy to replicate.

These limitations reflect the current trade-offs in VR sports design. Which are balancing immersion and responsiveness while working within the constraints of hardware that wasn't built specifically for high-fidelity physics interactions.

4.4 Future Work

While the current implementation successfully delivers a functional and immersive basketball shooting experience, there are several promising directions for future development that could enhance both the realism and replayability of the project.

4.4.1 Dribbling Rework

Although an initial dribbling mechanic was attempted the outcome proved too inconsistent for reliable gameplay. Future versions could integrate advanced hand tracking or machine learning-based gesture detection to better recognize dribbling motions. Incorporating controller haptics when the ball is bounced down could also improve immersion, as would a more robust bounce detection system using predictive physics. Another avenue would be to

develop simplified dribble assists, similar to the throwing assist, to improve rhythm and responsiveness while retaining a skill ceiling. It needs to be able to recreate the flair brought from real-life dribbling which will be the most difficult part to recreate since the movements are so complex and include many direction changes whilst the ball is still going down.

4.4.2 Expanded Game Modes

User feedback indicated interest in more types of modes beyond the existing Free Play, Timed, and Shooting Challenge. They could include:

- Multiplayer mode, allowing players to play with or against online players, which would increase the social value and increase the competitiveness aspect.
- AI Opponent mode, using Unity's AI bot system and simple decision trees to simulate defending or scoring behaviour.
- Skill training drills, focusing on accuracy, speed, or movement under pressure to mimic professional practice routines.
- Arcade challenges, such as moving hoops or combo-based scoring, which could add a more playful twist.

These would certainly elevate the fun aspect of the game increasing replayability and interest in the experience. This would almost double the size of the project because now not only offense would be played but also defence and the interaction between both.

4.4.3 Visual and Environmental Improvements

Some cosmetic improvements were suggested such as introducing other courts or including a crowd with noise that reacts to how the game is going. The addition of music would help with enriching the game feel. Lastly, being able to customize your court, ball and avatar would help the experience feel more personal.

4.5 Conclusion

This chapter presented a comprehensive evaluation of the VR basketball prototype, drawing on user feedback, observational analysis, and performance testing. The results confirm that the main objectives (creating an immersive and fun basketball shooting experience) were met successfully. While immersion and enjoyment were rated highly by users, limitations in VR controller fidelity and the complexity of replicating real-life dribbling were challenging. Nonetheless, these findings offer a clear direction for future development, including more refined mechanics, expanded game modes, and polishing. The feedback gathered has helped validate the strengths of the implementation while highlighting meaningful areas for improvement.

List of References

1. Lee, S. *9 Game-Changing Applications of VR Training in Sports Analytics.* [Online]. Year. [01/04/2025]. Available from: https://www.numberanalytics.com/blog/9-game-changing-applications-vr-training-sports-analytics
2. Brownlee, T. *Virtual reality for sports training: Can VR help athletes?.* [Online]. Year. [01/04/2025]. Available from: https://www.scienceforsport.com/virtual-reality-for-sports-training/?srsltid=AfmBOorDIXCK6QcptX9aRaAbZzZcCzbciacgVEuHmfLqle0mKh9NbuDC
3. Sniffen K, Noel-London K, Schaeffer M, Owoeye O. Is Cumulative Load Associated with Injuries in Youth Team Sport? A Systematic Review. *Sports Med Open*. 2022;8(1):117. Published 2022 Sep 16. Available from: doi:10.1186/s40798-022-00516-w
4. Chan, C.H.M., Ma, M.K.I., Chan, T.K. *et al.* Current applications of virtual reality in basketball training: a systematic review. *Sports Eng* **27**, 26 (2024). Available from: https://doi.org/10.1007/s12283-024-00469-1
5. Wei, Wenting, Qin, Ziqi, Yan, Bihong, Wang, Qiulin, Application Effect of Motion Capture Technology in Basketball Resistance Training and Shooting Hit Rate in Immersive Virtual Reality Environment, *Computational Intelligence and Neuroscience*, 2022, 4584980, 9 pages, 2022. Available from: https://doi.org/10.1155/2022/4584980
6. Sports Business Journal. *VReps Uses Virtual Reality as a Basketball Training Tool, Turning Playbooks Into Interactive Video Games.* [Online]. Year. [03/04/2025]. Available from: https://www.sportsbusinessjournal.com/Daily/Issues/2022/07/28/Technology/vreps-virtual-reality-basketball-training-playbooks-video-games/
7. VReps. *Jalen Williams, the 12th overall pick by the Oklahoma City Thunder in the first round of the 2022 NBA Draft, tests out VReps*. [Online]. Year. [03/04/2025]. Available from: https://www.sportsbusinessjournal.com/Daily/Issues/2022/07/28/Technology/vreps-virtual-reality-basketball-training-playbooks-video-games/
8. Wan-Lun, T. Training Assistant : *Basketball Tactic and Decision-making Training VR System*. [Online]. Year. [04/04/2025]. Available from: https://mislab.cs.nthu.edu.tw/project-pages/basketball-tactic-vr-system.html

9. Gym Class. *About Gym Class.* [Online]. Year. [04/04/2025]. Available from: https://www.gymclass.com/#:~:text=The%20company%27s%20mission%20is%20to,licensing%20relationship%20with%20the%20NBA.&text=Download%20and%20play%20Gym%20Class,Kit%20for%20more%20info%20&%20assets.&text=%E2%80%8DWorking%20at%20Gym%20Class,his%20previous%20startup%20to%20Coinbase
10. Gedeon, K. *Kevin Durant-backed basketball game 'Gym Class VR' launches on Meta Quest — why it feels so 'realistic'*. [Online]. Year. [05/04/2025]. Available from: https://library.leeds.ac.uk/referencing-examples/10/leeds-numeric/754/website-or-webpage
11. Reality Remake. *3 Best VR Basketball Games on the Oculus Quest 2.* [Online]. Year. [05/04/2025]. Available from: https://www.realityremake.com/articles/3-best-vr-basketball-games-on-the-oculus-quest-2
12. WhiteSword. *Blacktop Hoops: Analysis.* [Online]. Year. [05/04/2025]. Available from: https://www.realovirtual.com/articulos/6603/blacktop-hoops-analisis#google_vignette
13. F. P. Brooks, "What's real about virtual reality?," in *IEEE Computer Graphics and Applications*, vol. 19, no. 6, pp. 16-27, Nov.-Dec. 1999, Available from: doi: 10.1109/38.799723
14. M. T. Palmer, ‘‘Interpersonal communication and virtual reality: Mediating interpersonal relationships,’’ in Communication in the Age of Virtual Reality, F. Biocca and M. R. Levy, Eds. Hillsdale, NJ, USA: Lawrence Erlbaum, 1995, pp. 277–302. Available from: https://www.taylorfrancis.com/books/mono/10.4324/9781410603128/communication-age-virtual-reality-frank-biocca-mark-levy
15. EyeWiki. *Stereopsis and Tests for Stereopsis.* [Online]. Year. [06/04/2025]. Available from: https://eyewiki.org/Stereopsis_and_Tests_for_Stereopsis#References
16. ScienceDirect. *Stereopsis.* [Online]. Year. [06/04/2025]. Available from: https://www.sciencedirect.com/topics/neuroscience/stereopsis#:~:text=Stereopsis%20refers%20to%20the%20perception,Encyclopedia%20of%20the%20Eye%2C%202010
17. Coursera. *Six Degrees of Freedom Explained.* [Online]. Year. [06/04/2025]. Available from: https://www.coursera.org/articles/six-degrees-of-freedom
18. Barnard, D. *Degrees of Freedom (DoF): 3-DoF vs 6-DoF for VR Headset Selection .* [Online]. Year. [06/04/2025]. Available from: https://library.leeds.ac.uk/referencing-examples/10/leeds-numeric/754/website-or-webpage
19. Meta. *Learn how headset tracking works on Meta Quest.* [Online]. Year. [08/04/2025]. Available from: https://www.meta.com/en-gb/help/quest/598701621088668/

20. Zindulka, T., Bachynskyi, M., & Müller, J. (2020, April). Performance and experience of throwing in virtual reality. In Proceedings of the 2020 CHI conference on human factors in computing systems (pp. 1-8). Available from: https://doi.org/10.1145/3313831.3376639
21. Amirpouya Ghasemaghaei, Mykola Maslych, Yahya Hmaiti, Esteban Segarra Martinez, and Joseph J. LaViola Jr.. 2024. Towards Better Throwing: A Comparison of Performance and Preferences Across Point of Release Mechanics in Virtual Reality. In Graphics Interface (GI '24), June 03–06, 2024, Halifax, NS, Canada. ACM, New York, NY, USA, 11 pages. Available from: https://doi.org/10.1145/3670947.3670963
22. Magevo Studio. *Making a Basketball Court in Blender 2.8 (Timelapse).* [Online]. Year. [02/02/2025]. Available from: https://www.youtube.com/watch?v=tSZUhgul_l4
23. Rider University. *VR: Virtual Reality- Equipment & Safety.* [Online]. Year. [12/05/2025]. Available from: https://guides.rider.edu/vrsafety#:~:text=Controllers%3A,box%3B%201%20for%20each%20controller
24. Unity. *Unity User Manual 5.5.* [Online]. Year. [02/03/2025]. Available from: https://docs.unity3d.com/550/Documentation/Manual/class-Rigidbody.html
25. Unity. *Unity User Manual 5.5.* [Online]. Year. [02/03/2025]. Available from: https://docs.unity3d.com/Manual/static-batching.html
26. Williams, B. *NBA changing format of All-Star 3-point contest*. [Online]. Year. [20/04/2025]. Available from: https://www.sportbusiness.com/news/nba-changing-format-of-all-star-3-point-contest/

Appendix A
Self-appraisal

A.1 Critical self-evaluation

Overall, the project achieved all its main goals even if a mechanic wasn't finalised or implemented the experience was still functional, immersive and enjoyable which was the main point of the project.

However, some aspects of the project could have been managed more effectively. Early feature planning was probably overly ambitious, specifically around complex mechanics like dribbling. Better scoping and prioritisation could have reduced time spent on less crucial features and allowed greater focus on polish and user experience refinement.

Whilst version control was used, it wasn't from the get go so the commit history isn't as structured or descriptive as it should've been. The testing phase also could've included a broader range of users for a more diverse feedback but overall the testing was still relevant and useful.

Overall execution was successful but some areas in project management and scope definition could've been done better.

A.2 Personal reflection and lessons learned

At the tail end of the projects development and specially during writing the report, the sheer amount of work was really piling up due external factors and other modules and their required attention time so, project management wise I needed to spread out the time a lot more effectively and realistically and not expect me to be able to keep such a rigorous and intense routine of working all day weeks on end.
Scope management needs to be taken into consideration as shown by how I didn't really expect a mechanic that is commonly implemented in all basketball VR games, dribbling, to be so hard to replicate. Leaving certain aspects to the side even if at some point they were deemed essential, to in turn focus more on real achievable goals to ensure the success of the project, was an important lesson I learned.

Early prototyping, testing and preparing for the future of the project is also essential as it is the case with any software project. Anything you start building has value so its important to keep track of progress always otherwise its really easy to get lost in your own ideas and at

some point you don’t have it documented to show which is really, at the end of the day, the point of writing a report.

VR and any other type of software development has its trade-offs. It is really important to find a balance between the realism and the gameplay and understand that hardware limitations are very much real and understanding how to work around too.

Communication with users is crucial and I realised it when first letting people test my early prototypes and noticing them struggling a lot. Obviously I didn’t struggle because at that point I would’ve practiced those same shots hundreds of times during testing so I already had the “hang of it”. Understanding what people struggled with and that the point of the project wasn’t to build something only I enjoyed or could easily interact with, was an important lesson learned. The point of the project was to build something that anyone can pick up, understand and have fun with so their feedback and listening to criticism is really important.

A.3 Legal, social, ethical and professional issues

A.3.1 Legal issues

Legal considerations included:

- **Copyright and Licensing:** All 3D models, textures, and sounds were sourced from either the Unity Asset Store or other libraries that explicitly allow free use for personal or academic projects. Proper documentation and attribution were used.
- **User Data Privacy:** Every person that participated in the testing process had their identities kept confidential and their consent was properly disclosed through a signed consent form after they’ve read the participant information sheet. No identifiable or personal information was stored.
- **Software Licenses:** Unity was used under its free license for academic purposes. All third-party libraries or assets were checked to ensure compatibility with non-commercial use.

A.3.2 Social issues

Social considerations primarily concerned inclusivity and accessibility:

- **Accessibility:** The VR experience assumed a certain physical ability (e.g., standing upright, arm movement). While common in VR, future versions could explore accessibility settings (e.g., seated mode, adjustable court height, different types of shot assists).

- **Social Impact of VR:** By providing an engaging sports simulation in a small physical space, the project contributes to broadening participation in community games and activities that could range from just chatting online to playing pickup basketball at the park.

A.3.3 Ethical issues

Ethical considerations included:

- **Participant Consent:** Informed consent was obtained before user testing, with clear options for participants to withdraw at any time.
- **Health and Safety:** VR experiences can cause motion sickness or discomfort. Testers were advised to stop at any point if they felt unwell, and the application itself was designed to minimize any type of nauseating movements or environments.
- **Honest Reporting:** All user testing results and project outcomes are reported honestly, without manipulation to achieved a perceived success.

A.3.4 Professional issues

Professional standards were followed throughout the project, including:

- **Version Control Practices:** Use of GitHub ensured code was consistently backed up and changes were transparent and reversible.
- **Documentation:** Clear code commenting, external diagrams, and structured reporting helped maintain a professional standard of work.
- **Adherence to Best Practices:** Development practices followed Unity’s and Oculus’s recommended guidelines for VR.

Appendix B

External Materials

Rim sound - https://www.youtube.com/watch?v=Qccfpnl-wLk

2nd rim sound - https://www.youtube.com/watch?v=QWP_fG3PiI8

Dribble sounds - https://pixabay.com/sound-effects/basketball-99685/

Appendix C

Test Case Table

Feature	Test Case	Result
Throw detection	Fast throw and soft lob	Pass
Scoring logic	Ball enters from above and counts	Pass
Scoring logic	Ball enters from under and doesn’t count	Pass
Menu functionality	Select game modes and go back to menu	Pass
UI updates	Score + FG% after shot	Pass
Audio System	Ball bounce triggers sound on different materials	Pass
Ball Spawn System	Ball spawns when press input but not when holding	Pass
Ball Spawn System	Sticking hand outside of the invisible bounds and trying to spawn a ball out of limits	Fail
Ball Spawn System	Spawns 6th ball (oldest removed)	Pass
Ball Spawn System	Balls can’t be spawned outside of a game mode	Fail
Correct shot detection	Shoot from midrange and from three point range and check scoreboard	Pass
Sound logic	Made swish shot triggers the correct audio	Pass
Dribbling System	Downward hand movement on the ball produces a bounce	Partial

Appendix D

User Test Consent Form

University of Leeds

Consent Form (User Testing)

Title of Project: Creating an Immersive Basketball Experience in Virtual Reality

Name of Project Student: Pio Canas Toimil

Initial the box if you agree with the statement to the left

1 I confirm that I have read and understand the information sheet dated [/ /] explaining the above project and I have had the opportunity to ask questions about the project. ☐

2 I understand that my participation is voluntary and that I am free to withdraw at any time without giving any reason and without there being any negative consequences. In addition, should I not wish to answer any particular question or questions, I am free to decline. *Insert contact details here of project student.* ☐

3 I understand that my responses will be kept strictly confidential. I understand that my name will not be linked with the project materials, and I will not be identified or identifiable in the report or reports that result from the project. ☐

4 I agree for the data collected from me to be used in future research. ☐

5 I agree to take part in the above project and will inform the project student should my contact details change. ☐

Name of participant (*or legal representative*)	Date	Signature
Project student	Date	Signature

To be signed and dated in presence of the participant

Appendix E

User Test Information Sheet

Project Information Sheet

Project Title: Creating an Immersive Basketball Experience in Virtual Reality

You are being invited to take part in a student project. Before you decide, it is important for you to understand the aim of the project and what participation will involve. Please take time to read the following information carefully and discuss it with others if you wish. Ask if there is anything that is not clear or if you would like more information. Take time to decide whether or not you wish to take part. Thank you for reading this.

Project Aim: This project explores the development of a virtual reality (VR) basketball game using the Meta Quest 2 headset. The main aim is to evaluate how immersive and realistic the gameplay feels—specifically, how closely shooting and other basketball mechanics in VR replicate real-world actions. The project will run from October 2024 to April 2025

Why have I been chosen? You have been selected to take part in this study because you are within the target audience for VR sports games. Participants will include people with varying levels of experience in basketball and VR gaming, to provide a wide range of feedback. In total, around 3 to 5 participants will be involved in this short user testing phase.

Do I have to take part?
It is up to you to decide whether or not to take part. If you do decide to take part you will be given this information sheet to keep (and be asked to sign a consent form) and you can still withdraw at any time. You do not have to give a reason.

What will happen to me if I take part? If you choose to participate, you will be asked to play a short demo of the VR basketball game on the Meta Quest 2 headset. The session will last approximately 10–15 minutes. During gameplay, observations may be made regarding your interaction with the game, such as shooting mechanics or difficulty using the UI.
After the session, you will be asked to complete a short feedback form that includes both rating-scale questions and optional written responses. This will help evaluate realism, immersion, ease of use, and enjoyment.

Will my taking part in this project be kept confidential? No personal details will appear in any written reports. Notes will only be used for academic purposes and will not be shared with anyone outside of the University. No images or videos will be taken.

What type of information will be sought from me and why is the collection of this information relevant for achieving the project's objectives? You will be asked to provide your opinion on various aspects of the VR basketball experience, such as how realistic the throwing feels, how intuitive the controls are, and whether the game was enjoyable. This feedback is crucial for evaluating the effectiveness of the gameplay design and identifying areas for improvement in immersion and usability.

What will happen to the results of the project?
The results of this project will be published in a report to be submitted for assessment at the end of the undergraduate module COMP3931 Individual Project in the School of Computer Science at the University of Leeds.

Contact for further information:
Pio Canas Toimil
Undergraduate Student, School of Computer Science
University of Leeds
e-mail: sc22pct@leeds.ac.uk

If you decide to participate in this project, you will be given a copy of this information sheet and a signed consent form to keep. Thank you very much for taking the time to read this information sheet.